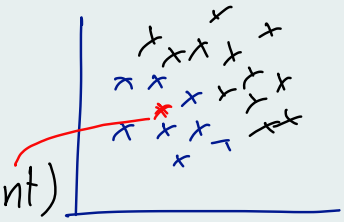
Limitations with KNN :

Failure case 1 :

Large datasets → (5L, 100)

- If the dataset is large.
- KNN is a Lazy learning technique.
  - means nothing will happen in training phase everything will be done in predicting / testing phase.
- Exact process will start at the time of query point (new data point)



query point → prediction

Training → nothing.

- In training phase it will just store the distances.
- So in prediction it will start finding the distance, sorting and find the nearest one
- It will take much time if dataset is large.

5L distance → sorting → majority

2. High dimesional data :

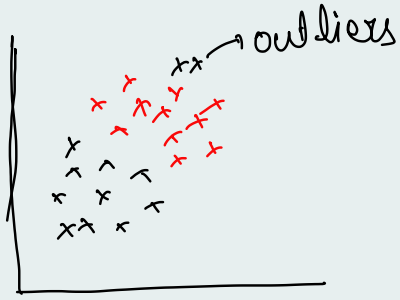
- If the no. of features increase

  curse of dimen → distance concept is not reliable.

- KNN is totally works on distance concept. If the no. of dimension is high distance will not reliable

3. Outliers :

- Query point will deviate with outlier

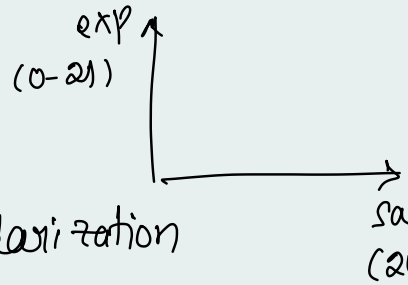


- KNN is sensitive to outliers.

4. Non-homogenous feature scales :

- If the diff scales of data should apply standarization

exp | salary



5. Imbalanced dataset

- KNN will be baised towards one class.

6. Inference not for prediction

- KNN will do prediction well but can not get

any inference from data.

Like $x_1, x_2 \longrightarrow Y$

we can not understand x_1 or x_2 which contribute more on 'y'. we can not find any relationship any kind of insights in training phase.

Distances:

1. Euclidian Distance

$P_1(x_1, y_1)$ & $P_2(x_2, y_2)$

$$E\text{-}D = \sqrt{(x_2-x_1)^2+(y_2-y_1)^2}$$

2. Manhattan Distance $= |x_2-x_1| + |y_2-y_1|$

3. Minkowski Dist $= \sqrt[h]{(x_2-x_1)^h+(y_2-y_1)^h}$

4. Cosine Similarity $= \frac{AB}{\|A\|\|B\|} = \frac{\Sigma A \cdot B}{\sqrt{\Sigma A^2} \cdot \sqrt{\Sigma B^2}}$

Input $\in \mathbb{R}^{d-1}$ → dimension

→ Real/cont num values

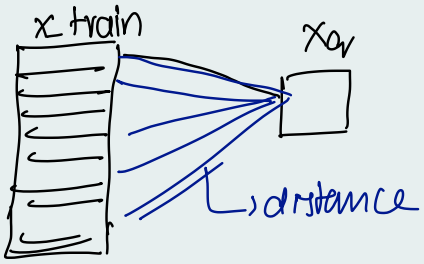
Total dataset is (nxd) shape

Q1: What should be the value of 'K'?

Q2: Which distance matrix should be used?

→ For a given query point find 'K' nearest neighbours

How → distance



Variance

Dataset

Train Test Split

diff random states

↓

Diff rand Samples

Train & Test

Rep sample pop^{n}

↓

Dataset

Train data

↓

Rep pop^{n}

↓

Test data

Tr_1 Te_1 → M_1 — $Perf_1$

Tr_2 Te_2 — M_2 — $Perf_2$

Tr_3 Te_3 — M_3 — $Perf_3$

⋮ | — .

Tr_n Te_n — M_n — $Perf_n$

Expected

All perf

same

↓

Models

Reality

There is a var in the perf

Models

As Train data changes

↳ Model changes

↳ Var in pred

Model varies by having a small change in train data

⇒ Variance

$x = (5.1, 3.5)$

$\hat{x} = (6.2, 2.9)$

$$\text{distance}(x, \hat{x}) = \sqrt{(6.2-5.1)^2 + (2.9-3.5)^2}$$

$$= \sqrt{(1.1)^2 + (0.6)^2}$$

$$=$$

Steps to build KNN

1. Choose a value for 'K'
2. Compute the distance btw each data point with query point (x_q)
   - Euclidean
   - Manhattan.

$D = [D_1, D_2, D_3, \ldots D_n]$

3. Find K-nearest data points based on distances.

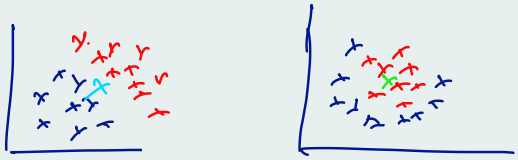
[0, 0, 1, 1, 0]

4. Choose highest majority of class.

$\therefore x_q \rightarrow 0$.

Points to be noted:

1. KNN is a distance based algorithm.
2. Actual implementation of KNN algo starts during the prediction phase
3. In the training phase the algo only saves the data.
4. KNN is the memory based algorithm.
5. In KNN, prediction time is more than the training time.
6. KNN work for both non-linear & linear data.



7. KNN requires scaling.
8. KNN is non-parametric Algo

- No assumptions required to build KNN algo, the data can be any distribution.

9. KNN is the algorithm that is used for "Imputing Missing Value"